KUBERNETES LAB · HPCLUSTER · AP-SOUTH-1

Sharing T4 GPUs on EKS

Make one NVIDIA GPU serve several pods. You install the NVIDIA device plugin, then compare two ways of sharing a GPU: time-slicing and MPS, which splits the GPU's compute units (SMs) between pods.

`EKS 1.36` `2 × g4dn.xlarge` `Tesla T4 · 15360 MiB · 40 SMs`
`Driver 580.178 · CUDA 13.0` `nvidia-device-plugin 0.20.1`

0 Cluster analysis

Findings from the cluster on 6 October 2026, before any GPU setup:

ITEM	FINDING	IMPACT
GPU nodegroup	`gpunode` : 2 × g4dn.xlarge (4 vCPU, 16 GiB RAM, 29 pods max)	2 physical GPUs in total
GPU	1 × Tesla T4 per node (Turing)	No MIG support. Sharing must use time-slicing or MPS.
Node image	`AL2023_x86_64_NVIDIA` 1.36.4-20260930	The driver and container toolkit are already installed, so you don't need the GPU Operator.
Node labels	`nvidia.com/gpu.present=true` set by the AMI	The device plugin's node affinity already matches
`nvidia.com/gpu` capacity	Missing	The device plugin isn't installed, so GPU pods stay Pending
Taints	None on GPU nodes	Ordinary pods can land on GPU nodes (see step 6)
Existing GPU pods	`dcgm-server` DaemonSet (EKS node monitoring agent)	Monitoring only. Leave it in place. It is handy for running `nvidia-`

```
smi .
```

1 Concepts

The device plugin advertises **N replicas** of `nvidia.com/gpu` for each physical GPU. The scheduler only sees the count. The sharing mechanism decides what happens on the GPU itself.

Time-slicing · 4 replicas



Pods take turns on the whole GPU. The horizontal axis is time.

MPS · 4 replicas



Pods run at the same time, each on about 10 of the 40 SMs. The horizontal axis is SMs.

	EXCLUSIVE	TIME-SLICING	MPS	MIG
Mechanism	One pod owns the GPU	The GPU context-switches between pods	One MPS server merges CUDA contexts so kernels run in parallel	Hardware partitions
Compute isolation	Full	None	SM cap per pod: 100/replicas %	Full
Memory isolation	Full	None. One pod can exhaust memory for the others.	Limit per pod (≈ total/replicas)	Full
Fault isolation	Full	Shared	Shared	Full
On T4	Yes	Yes	Yes (Volta and newer)	No (A100, H100 and similar only)

Fits	Training	Dev, notebooks, bursty inference	Many small, steady inference jobs	Multi-tenant production

2 Baseline: GPUs aren't schedulable yet

```
kubectl get nodes -L node.kubernetes.io/instance-type,eks.amazonaws.com/nodeg
kubectl get nodes -o custom-columns=NAME:.metadata.name,GPU:.status.allocatable.nvi
# GPU column shows <none>

kubectl apply -f 01-gpu-smoke-test.yaml
kubectl get pod gpu-smoke-test            # Pending
kubectl describe pod gpu-smoke-test | grep -A3 Events
# 0/3 nodes are available: ... Insufficient nvidia.com/gpu
kubectl delete -f 01-gpu-smoke-test.yaml
```

Copy

The driver works. Only Kubernetes can't see the GPU yet:

```
kubectl -n kube-system exec ds/dcgm-server -- nvidia-smi
```

Copy

3 Install the NVIDIA device plugin

This installs the device plugin, GPU Feature Discovery (GFD) and Node Feature Discovery. The values file defines three sharing profiles: `exclusive`, `time-slicing` (the default) and `mps`.

```
helm repo add nvdp https://nvidia.github.io/k8s-device-plugin
helm repo update

helm upgrade -i nvdp nvdp/nvidia-device-plugin \
  --namespace nvidia-device-plugin --create-namespace \
  --version 0.20.1 \
  -f nvdp-values.yaml

kubectl -n nvidia-device-plugin get pods -o wide
```

Copy

Expect one `nvidia-device-plugin` pod and one `gpu-feature-discovery`

pod on each GPU node, plus the NFD master, worker and gc pods. The `mps-control-daemon` has no pods yet. It only runs on nodes labelled `nvidia.com/mps.capable=true`.

```
kubectl get nodes -l eks.amazonaws.com/nodegroup=gpunode \
  -o custom-columns=NAME:.metadata.name,GPU:.status.allocatable.nvidia\\.com/gpu,\
STRATEGY:.metadata.labels.nvidia\\.com/gpu\\.sharing-strategy,\
REPLICAS:.metadata.labels.nvidia\\.com/gpu\\.replicas,\
PRODUCT:.metadata.labels.nvidia\\.com/gpu\\.product
# GPU=4  STRATEGY=time-slicing  REPLICAS=4  PRODUCT=Tesla-T4-SHARED
```

Copy

Checkpoint. Each node now reports 4 GPUs. The product label gets a `-SHARED` suffix so workloads can tell shared GPUs apart.

4 Time-slicing lab

Get a single-pod baseline first, then share the GPU between 8 pods on 2 physical GPUs.

```
kubectl apply -f 02-timeslicing-demo.yaml
kubectl scale deploy gpu-timeslice-demo --replicas=1
kubectl logs -f deploy/gpu-timeslice-demo     # note billion interactions per secon

kubectl scale deploy gpu-timeslice-demo --replicas=8
kubectl get pods -l app=gpu-timeslice-demo -o wide
# 8/8 Running, 4 on each node

# Look at the GPU from the dcgm-server pod on the same node
kubectl -n kube-system get pods -o wide | grep dcgm
kubectl -n kube-system exec <dcgm-server-pod> -- nvidia-smi   --query-gpu=utilizati
# 100 %, ~250 MiB, ~70 W, Default

for p in $(kubectl get pods -l app=gpu-timeslice-demo -o name); do kubectl logs $p
```

Copy

Scale past capacity. Scheduling stops at 2 nodes × 4 replicas:

```
kubectl scale deploy gpu-timeslice-demo --replicas=10
```

Copy

```
kubectl get pods -l app=gpu-timeslice-demo | grep Pending    # 2 Pending
kubectl delete -f 02-timeslicing-demo.yaml
```

Discuss. Memory isn't limited. One pod can allocate all 15 GiB and make the other three fail.

5 MPS lab: splitting the GPU's SMs

Switch the second node to the `mps` profile with a label. You don't need to reinstall.

Copy

```
NODE_B=<second-gpu-node>   # kubectl get nodes -l eks.amazonaws.com/nodegrou
kubectl label node $NODE_B nvidia.com/device-plugin.config=mps --overwrite

# Wait 30-60 s for GFD to relabel the node and the MPS control daemon to start
kubectl get node $NODE_B -L nvidia.com/gpu.sharing-strategy,nvidia.com/mps.capable,
# mps   true   4
kubectl -n nvidia-device-plugin get pods -o wide | grep mps-control-daemon
```

Copy

```
kubectl apply -f 03-mps-demo.yaml
kubectl get pods -l app=gpu-mps-demo -o wide        # 4 pods on node B

# Pods only get the MPS pipe. The limits are set as defaults on the MPS daemon:
kubectl exec deploy/gpu-mps-demo -- env | grep CUDA_MPS
# CUDA_MPS_PIPE_DIRECTORY=/mps/nvidia.com/gpu/pipe
kubectl -n nvidia-device-plugin logs ds/nvdp-nvidia-device-plugin-mps-control-daemo
# set_default_device_pinned_mem_limit 0 3840M   about 1/4 of 15360 MiB
# set_default_active_thread_percentage 25       about 10 of 40 SMs

# The GPU switches to Exclusive_Process: 1 MPS server and 4 clients
kubectl -n kube-system exec <dcgm-server-pod-on-node-B> -- nvidia-smi --query-gpu=c
kubectl -n kube-system exec <dcgm-server-pod-on-node-B> -- nvidia-smi --query-compu

for p in $(kubectl get pods -l app=gpu-mps-demo -o name); do kubectl logs $p | tail

# Show the SM cap: a lone MPS pod still cannot use the whole GPU
kubectl scale deploy gpu-mps-demo --replicas=1
kubectl logs -f deploy/gpu-mps-demo
```

Known issue, chart 0.20.1. The `mps-control-daemon` pod shows `1/2` because its sidecar keeps restarting with `error finding pid: no process found`. The MPS daemon container is healthy and MPS pods work, so you can ignore it for the lab.

Compare. Compare interactions per second in step 4 and step 5 against the results table below. MPS pods run kernels at the same time with no context switching. Each pod also has a guaranteed limit on its share of SMs and memory.

Clean up:

```
kubectl delete -f 03-mps-demo.yaml
kubectl label node $NODE_B nvidia.com/device-plugin.config-     # back to time-slic
# To turn sharing off on a node: nvidia.com/device-plugin.config=exclusive
```

Copy

R Measured results

Measured on hpcluster on 6 October 2026. Tesla T4 running `nbody -benchmark -numbodies=128000 -i=200`, averaged over 3 or 4 runs per pod.

SETUP	PER POD	TOTAL PER GPU	VS. 1 POD ALONE
1 pod, whole GPU	216.3	216.3	100%
Time-slicing, 2 pods per GPU	~95	~190	88%
Time-slicing, 4 pods per GPU	~48	192.0	89%
MPS, 4 pods per GPU	~53	212.6	98%
MPS, 1 pod alone (25% SM cap)	75.7	75.7	35%

Figures are billions of nbody interactions per second.

- Time-slicing loses about 11% to context switching. MPS loses about 2% because kernels run side by side.
- An MPS pod stays capped even when the GPU is otherwise idle. It scores 35% rather than 25%, most likely because the T4 boosts its clock higher under light

load. The cap is what guarantees each pod its share.

6 Shared training cluster setup

Keep ordinary pods off GPU nodes

```
aws eks update-nodegroup-config --cluster-name hpcluster --nodegroup-name gpu \
  --region ap-south-1 \
  --taints 'addOrUpdateTaints=[{key=nvidia.com/gpu,value=true,effect=NO_SCHEDULE}]'
```

Copy

The device plugin chart and all lab manifests already tolerate this taint.

Limit GPUs per student namespace

2 nodes × 4 replicas gives 8 shared GPUs for the whole class.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: gpu-quota
spec:
  hard:
    requests.nvidia.com/gpu: "1"
```

Copy

Capacity and cost

- A g4dn.xlarge allows 29 pods, and DaemonSets already use about 10. Keep replicas per GPU at about 10 or fewer.
- GPU nodes are billed on demand. Scale `gpunode` to 0 when the lab ends.

7 Troubleshooting

SYMPTOM	CHECK
`nvidia.com/gpu` still missing	`kubectl -n nvidia-device-plugin logs ds/nvdp-nvidia-`

	`device-plugin -c nvidia-device-plugin-ctr`
GPU count unchanged after relabel	`kubectl -n nvidia-device-plugin logs ds/nvdp-nvidia-device-plugin -c nvidia-device-plugin-sidecar`
MPS daemon not scheduled	The node has no `nvidia.com/mps.capable=true` label. Check the GFD pod logs.
CUDA errors that mention MPS	`kubectl -n nvidia-device-plugin logs ds/nvdp-nvidia-device-plugin-mps-control-daemon`
A pod requesting `nvidia.com/gpu: 2` is rejected	This is expected. `failRequestsGreaterThanOne` blocks it because 2 replicas don't mean twice the GPU.
Pods Pending with `Too many pods`	The 29-pod limit of g4dn.xlarge is full of non-GPU pods. Taint the GPU nodes (step 6).
Helm install times out, pods Pending	Check `kubectl get nodes` . The node groups may be scaled to 0 or cordoned.

A Lab files

▼ nvdp-values.yaml

Copy

```
# Helm values for nvdp/nvidia-device-plugin 0.20.1
# Cluster: hpcluster (ap-south-1) | GPU nodegroup: gpunode (2 x g4dn.xlarge, 1 x

# GPU Feature Discovery: adds nvidia.com/gpu.* labels and nvidia.com/mps.capable
# which the MPS control daemon needs. It also deploys Node Feature Discovery (NF
gfd:
  enabled: true

# Run only on the GPU nodegroup
nodeSelector:
  eks.amazonaws.com/nodegroup: gpunode

# Named sharing profiles. Pick one per node with the label
#   nvidia.com/device-plugin.config=<profile>
config:
  default: time-slicing
  map:
```

```
    # No sharing: 1 pod per GPU (baseline)
    exclusive: ├─
      version: v1
      flags:
        migStrategy: none

    # Time-slicing: 4 pods take turns on the whole GPU.
    # No memory or fault isolation.
    time-slicing: ├─
      version: v1
      flags:
        migStrategy: none
      sharing:
        timeSlicing:
          renameByDefault: false
          failRequestsGreaterThanOne: true
          resources:
            - name: nvidia.com/gpu
              replicas: 4

    # MPS: 4 pods run kernels at the same time.
    # Each pod is limited to 100/4 = 25% of SMs and about 1/4 of GPU memory.
    mps: ├─
      version: v1
      flags:
        migStrategy: none
      sharing:
        mps:
          renameByDefault: false
          resources:
            - name: nvidia.com/gpu
              replicas: 4
```

▶ 01-gpu-smoke-test.yaml

▶ 02-timeslicing-demo.yaml